

Advanced Terraform on Azure



Student Manual

Advanced Terraform on Azure

Student Manual

Welcome to Advanced Terraform on Azure.....	5
Module 0 – Course Philosophy: Operating Terraform in Real Organisations	6
Purpose	6
Course Philosophy.....	6
Module 1: State Churn and Resource Identity	7
Module Overview.....	7
Terraform State and Resource Identity	8
What Is Churn?.....	8
Understanding Terraform Collection Types.....	9
Why count and lists can cause problems	10
Introducing for_each: Fixing Identity	10
Example code.....	11
Churn in Summary	12
Key Lessons.....	13
Module 2: Input Hygiene & Validation.....	13
Module Overview.....	13
Spotting the Problem.....	14
Using Locals in Terraform.....	14
Why Not Just Use Variables?	15
What Locals Are Good For.....	15
Why Locals Help Reduce Churn.....	16
Things Locals Are Not For	16
Validation and Guardrails	16
Variable Validation.....	16
Preconditions and Postconditions.....	17
Conditional Logic	18
Bringing It All Together.....	18
Takeaways	19
Module 3: Dynamic Blocks & Deterministic Naming	19
Module Overview.....	19

Starting Point: Module 2 Exemplar	19
Dynamic Blocks and for_each: Recap	20
Normalizing Subnet Inputs	20
Refactoring Subnets to for_each	20
Associating NSGs with Subnets	21
Testing Stability: Reordering and Extending	21
Summary	22
Module 4: Remote State and Multi-Environment Operation in Terraform	22
Module Overview	22
Why Local State Breaks Down	22
Remote State as Shared Infrastructure	23
The Backend Chicken-and-Egg Problem	23
Why Backend Configuration Is Treated Literally	24
Environments Are Defined by State, Not Code	24
Human Error Is the Primary Risk	25
Backend Reconfiguration as a Deliberate Act	25
Scaling the Pattern Beyond the CLI	26
How This Module Connects Forward	26
Module Summary	26
Module 5: Modularization, Contracts and Validations	27
Module Overview	27
Why Terraform Modules?	27
Module Boundaries: Security and Network	28
Refactoring a Flat Configuration	28
Designing Module Contracts	28
Validation and Guardrails at Module Level	29
Wiring Modules from the Root	29
Zero-Diff Refactor Principle	30
Changing Addresses with moved and state mv	30
Evolving Modules Over Time	31
Looking Ahead to Multi-Environment Use	31
Module 6 – Publishing and Consuming Terraform Modules	32
Module Overview	32
From Refactoring to Reuse	32

Modules as Products.....	32
Understanding Module Contracts	32
Roots and Responsibility	33
Why Versioning Matters	33
Environments and Promotion	33
Promotion Is a Human Decision	33
Automation and Its Limits.....	34
Environment Ownership	34
Closing Thoughts.....	34
Module 7: Change Governance and Promotion Control in Terraform	34
Why This Module Exists.....	34
When Informal Governance Stops Working.....	35
Automation Is Fast — But It's Not Wise.....	35
Promotion Is Not a Technical Step	36
Intent, Approval, and Execution Are Different Things	36
Governance Emerges from Layers, Not Tools.....	36
When Pipelines Are Asked to Do Too Much	37
How Governance Fails in Practice	37
Trust Boundaries and Where Power Really Lives	38
Identity Is Not Just Authentication.....	38
Governance-of-Governance: The Uncomfortable Question	38
Break-Glass Is Not Failure — It's Honesty	38
Plans as Signals, Not Just Previews	39
Closing the Loop.....	39
Module 8: Consuming and Reconciling Workloads with a Centralised Landing Zone.....	40
Why Terraform State Operations Matter.....	40
Terraform State as an Ownership Boundary	40
Importing Existing Infrastructure into Terraform State	41
Generated Configuration Is a Starting Point, Not a Design.....	41
Refactoring Without Resource Re-creation	42
Remote Backends and Operational Safety.....	42
Centralisation and the Platform / Workload Split	42
Architectural Choices When Consuming a Landing Zone	43
Applying the Model in Practice	44

Module Summary.....	45
Appendix A - Bloom's Taxonomy Alignment.....	45
Bloom's Taxonomy Alignment	45
Remember & Understand.....	45
Apply	46
Analyse	46
Evaluate	46
Create / Synthesize	47
Level-7 Learning Outcome Alignment	47
Summary	47
Appendix B – Level-7 / UK Apprenticeship & HE Mapping	48
Level-7 Alignment (UK Apprenticeship & Higher Education Frameworks)	48
Level-7 Descriptor Alignment (FHEQ).....	48
How This Course Meets Level-7 Expectations.....	48
Alignment to Level-7 Apprenticeship (IfATE – KSBs)	50
QAA Computing Benchmark Alignment.....	51
Summary Statement (For Accreditation / QA Use)	51

Welcome to Advanced Terraform on Azure

This course is designed for practitioners who already use Terraform and want to use it well.

Most Terraform training focuses on how to create infrastructure. In real organisations, however, infrastructure already exists. Multiple teams work in shared environments, changes are governed, and mistakes have real consequences. Terraform is expected not just to build resources, but to support safe, deliberate change over time.

That is the context this course is built around.

Throughout the course, Terraform is treated as a state-driven system, not a scripting tool. You will learn how Terraform determines identity, how it decides whether to modify or replace resources, and why small design choices can have large downstream effects. The emphasis is on predictability, safety, and clarity of intent rather than speed or convenience.

As the modules progress, the focus shifts from individual configuration to shared operation. You will work with remote state, multiple environments, modular boundaries, and promotion workflows. Terraform plans are treated as decision artefacts, used to assess impact and risk before change is approved.

In the later modules, the course moves beyond greenfield scenarios and into realistic, existing cloud estates. You will explore how to bring existing resources under Terraform management, how to refactor safely, and how workloads adapt when a centrally managed platform or Landing Zone becomes authoritative.



This is not a course about shortcuts or clever syntax. It is a course about professional judgement. By the end, you should be comfortable not only writing Terraform, but operating it responsibly — understanding when to act, when to pause, and how to evolve infrastructure safely over time.

Module 0 – Course Philosophy: Operating Terraform in Real Organisations

Purpose

This module establishes the intellectual and professional context for the course. It explains how Terraform is used in mature, regulated environments and how the course approaches infrastructure, automation, and governance.

Course Philosophy

This course is designed to teach Terraform not as a scripting tool, but as a system for managing infrastructure ownership, change, and risk over time.

Most introductory Terraform training focuses on syntax and short-lived, greenfield scenarios: writing configuration, running terraform apply, and observing resources appear. While this approach is useful for learning fundamentals, it does not reflect how Terraform is used in mature, regulated, or long-lived cloud environments.

In real organisations, infrastructure already exists. Multiple teams operate concurrently. Environments are shared, governed, and audited. Decisions about change must be deliberate, reviewable, and reversible. Terraform is expected not only to *create* infrastructure, but to *preserve intent*, *control blast radius*, and *surface risk* clearly before execution.

This course is built around that reality.

From the outset, you will be taught how Terraform thinks: how identity is derived from state, how design decisions influence lifecycle behaviour, and why apparently small choices (such as collection types or naming conventions) can have far-reaching consequences. Rather than optimising for speed or convenience, the course consistently prioritises determinism, predictability, and safety.



As the course progresses, Terraform is reframed from a single-user tool into a shared system. Modules introduce concepts such as environment separation, remote backends, module contracts, promotion intent, and governance controls—not as abstract best practices, but as necessary responses to organisational complexity. Automation is treated as an execution mechanism, not a decision-maker. Human intent remains central.

The final module deliberately shifts focus away from new infrastructure creation and toward brownfield operation. You learn how to recover ownership of existing resources, refactor configuration safely, and reconcile historical workloads with centrally managed platform models. Terraform plans are positioned as governance artefacts that support informed decision-making, rather than as commands to be executed blindly.

By the end of the course, you should understand not only *how* to use Terraform, but when, why, and under what constraints it should be used. The aim is to develop practitioners who can reason about infrastructure systems holistically—balancing technical correctness, organisational boundaries, and risk—rather than simply producing working code.

Module 1: State Churn and Resource Identity

Module Overview

In this module, you will explore how Terraform tracks infrastructure, why unexpected change (known as churn) occurs, and how design decisions around identity directly influence Terraform’s behaviour.

This module is intentionally conceptual. Before learning how to write Terraform, it is essential to understand how Terraform thinks.

By the end of this module, you should be able to:

- Explain how Terraform identifies resources
- Distinguish between different types of churn
- Recognize design patterns that cause unnecessary change
- Understand why `for_each` and maps are preferred over `count` and lists

Terraform State and Resource Identity

Terraform does not manage “resources” in the abstract. It manages state objects.

Each resource declared in configuration is recorded in the state file using a resource address. Terraform uses this address as the identity of the resource. All lifecycle decisions—whether to create, update, or destroy—are based on whether Terraform believes an address still refers to the same thing.

If Terraform believes a resource’s identity has changed, it will destroy the old resource and create a new one. If Terraform believes identity is unchanged, it will attempt to update the resource in place.

What Is Churn?

- **Churn** is *unnecessary or unintended change* during a Terraform apply
- It occurs when Terraform’s **model of identity or intent is unstable**
- Churn is not always destructive — but it is always a **risk indicator**

Three kinds of churn

- **Configuration churn** – resources change in place
- **State churn** – resources are destroyed and recreated
- **Semantic drift** – resources keep the same identity but change meaning

Churn is unnecessary or unintended change during a Terraform apply.

Churn is not inherently destructive. However, it is always a risk indicator, because it suggests that Terraform is reacting to changes in structure rather than to deliberate changes in intent.

State Churn

State churn occurs when Terraform destroys and recreates resources unnecessarily. This typically happens when resource identity changes.

Configuration Churn

Configuration churn occurs when Terraform updates resources in place. Identity remains stable, but behaviour changes.

Semantic Drift

Semantic drift occurs when a resource keeps the same identity but now represents something different than originally intended.

Understanding Terraform Collection Types

Terraform provides three primary collection types: **lists**, **sets**, and **maps**. While they can all hold multiple values, they differ in how Terraform tracks identity, order, and uniqueness. These differences are not cosmetic — they directly influence how Terraform determines whether a resource should be created, updated, or destroyed.

Type	Ordered	Duplicates ?	Identity
list	Yes	Yes	Position (index)
set	No	No	Value
map	No	No (keys unique)	Key

A **list** is an ordered collection where elements are identified by their position (index). Lists allow duplicates, and Terraform treats a change in order as meaningful. This makes lists easy to work with, but potentially dangerous when order does not represent intent. Reordering a list can cause existing resources to adopt new values, leading to configuration churn or semantic drift.

Example...

```
variable "allowed_cidrs" { type = list(string) }  
allowed_cidrs = ["10.0.0.0/24", "10.1.0.0/24"]
```

A **set** is an unordered collection of unique values. Terraform identifies each element by its value rather than its position. Because order is ignored and duplicates are removed, sets are ideal when the value itself defines identity, such as CIDR blocks or resource names. However, sets cannot carry additional per-element configuration — each value stands alone.

Example...

```
variable "allowed_cidrs" { type = set(string) }  
allowed_cidrs = ["10.0.0.0/24", "10.1.0.0/24"]
```

A **map** is an unordered collection of key–value pairs. Identity is defined explicitly by the key, and configuration is provided by the value. This makes maps the most expressive and stable collection type when each element requires its own behaviour or attributes. Maps are the preferred structure when both identity and intent must be preserved across changes.

Example...

```
variable "allowed_cidrs" { type = map(number) }  
allowed_cidrs = {"10.0.0.0/24"=200,"10.1.0.0/24"=201}
```

Key Design Insight

Choosing the correct collection type is a design decision, not a syntax preference. Lists imply order and positional identity. Sets imply value-based identity without metadata. Maps allow full control over identity and behaviour. In Terraform, stable infrastructure comes from choosing a collection type that matches the meaning of the data being modelled.

Why count and lists can cause problems

When count is used with a list collection type, Terraform assigns identity based on ordinal position (index). Reordering lists can cause silent reassignment of meaning and configuration drift.

```
variable "allowed_cidrs" { type = list(string) }  
name = "allow-https-${count.index}"
```

count creates resources based on a numeric index (0,1,2,...). Each instance's address is tied to its position in a list. Inserting or reordering items shifts indexes and can cause Terraform to update values or even replace resources if a ForceNew field changes.

Introducing for_each: Fixing Identity

Sets: Identity Without Metadata

```
variable "allowed_cidrs" { type = list(string) }  
  
for_each = toset(var.allowed_cidrs)  
name = "allow-https-${replace(replace(each.key, "/", "-"), ".", "-")}"
```

for_each assigns each resource a key instead of an index. Keys provide semantic, stable identities such as CIDRs, names, or IDs. Reordering or inserting new CIDRs no longer affects existing resources.

Static Keys Alone Are Not Perfect

```
for_each = {for idx, cidr in var.allowed_cidrs : cidr => 200 + idx}  
name = "allow-https-${replace(replace(each.key, "/", "-"), ".", "-")}"  
priority = each.value
```

Even though `for_each` stabilizes identity, derived fields can still drift. For example, if you use `'200 + index'` as a priority, inserting a new CIDR in the middle of the sorted list shifts later indices, updating those priorities. This creates configuration churn even though identities remain stable.

Final Pattern: Explicit Maps

```
variable "allowed_cidrs" { type = map(number) }

for_each = var.allowed_cidrs
name      = "allow-https-${replace(replace(each.key, "/", "-"), ".", "-")}"
priority  = each.value
```

The cleanest approach is to provide explicit identity and values using a map. Keys provide stable identity, and values remain explicit, preventing configuration churn.

Example code

count examples

Variable: `"allowed_cidrs" { type = list(string) }`

Values: `["192.168.1.0/24", "10.1.0.0/24", "10.2.0.0/24"]`

Example 1: Using count in a way that may induce configuration churn

```
resource "azurerm_network_security_rule" "allow_https" {
  count = length(var.allowed_cidrs)
  name  = "allow-https-${count.index}"
  priority = 200 + count.index
  source_address_prefix = var.allowed_cidrs[count.index]
}
```

This pattern ties resource identity to the index (`[0],[1],...`) but derives fields like `priority` and `source_address_prefix` from the list order. When you reorder or insert a new CIDR, Terraform keeps the same instance keys (so no replacements) but updates any fields whose values moved under those indexes. The result is noisy, order-driven update churn, even though nothing meaningful changed. Plans become harder to read and reviewers must re-verify innocuous diffs each time the list order changes.

Example 2: Using count in a way that may induce state churn

```
resource "azurerm_network_security_rule" "allow_https" {
  count = length(var.allowed_cidrs)
  name  = "allow-https-${replace(replace(var.allowed_cidrs[count.index], "/", "-"), ".", "-")}"
}
```

```

    priority = 200 + count.index
    source_address_prefix = var.allowed_cidrs[count.index]
}

```

Here the rule name is derived from the CIDR value. With count, the instance keys still stay the same on reorders, but the value at a particular index may change, which therefore also changes the name. Because name is a ForceNew field for NSG rules, Terraform must destroy and recreate those resources. This demonstrates state churn: replacements happen not because the intent changed, but because the configuration coupled a ForceNew attribute to a volatile, order-dependent value.

for_each example

variable: "allowed_cidrs" { type = map(number) }

values: {"10.1.0.0/24"=200,"192.168.1.0/24"=201,"10.2.0.0/24"=202}

Example

```

resource "azurerm_network_security_rule" "allow_https" {
  for_each = var.allowed_cidrs
  name     = "allow-https-${replace(replace(each.key, "/", "-"), ".", "-")}"
  priority = each.value
  source_address_prefix = each.key
}

```

Switching to this pattern changes identity from positional to a semantic key (the CIDR). Each rule's address is now ...["<CIDR>"], so reordering or inserting entries does not affect existing resources at all. By providing an explicit priority per CIDR in the map, you also eliminate order-derived updates—priorities no longer shift when new items are added. In practice, this yields deterministic, zero-diff plans for reorders and clean, surgical diffs when you add/remove a specific CIDR.

Churn in Summary

State churn = unwanted replacements caused by a change in resource identity (address) even though intent is unchanged. Configuration churn = updates only. Stable identities and explicit mappings eliminate both.

Key Lessons

Terraform's state identity drives replacement behaviour. `count` = fragile (identity by index). `for_each` = stable (identity by key). Changes to `ForceNew` fields (like `name`) always replace resources. Stable identities are essential for safe automation.

Lab 1

Configuration and State Churn

In this lab you will progress through churn inducing code, refactoring it iteratively to arrive at a business-ready zero churn configuration.



Module 2: Input Hygiene & Validation.



Module Overview

This module is about taking Terraform code that works but is fragile and hardening it to demonstrate why hygiene, determinism, and guardrails matter. Sloppily written code may plan successfully, but it often carries bad habits: messy keys, untidy CIDR lists, inconsistent casing, and no validation.

The goal here is not to ask “can Terraform create this cloud resource?”—Terraform will usually try its best with whatever input it’s given—but rather “can Terraform create the same resource predictably, even when humans type things differently?”

We achieve that by inserting a hygiene layer (`locals.tf`), normalising inputs, and enforcing rules that stop dangerous or inconsistent values from ever reaching the cloud.

Spotting the Problem

Our starting point in the lab for this module is an intentionally “messy” configuration: duplicate CIDRs (including 0.0.0.0/0), rule names copied straight from untidy keys, protocols in mixed case, and non-standardised tags. This proves a key point—Terraform will happily accept sloppy input unless we explicitly tell it otherwise.

Typical problems include:

1. Inconsistent capitalisation of protocols or directions
2. Untidy key names and rule identifiers
3. Duplicate or invalid CIDR blocks
4. Missing or inconsistent tagging

While this code might pass the planning phase, it’s fragile. Some issues only surface at apply time, and others (like an uppercase protocol) can cause even the plan to fail. However, intervention does not mean editing the inputs—they may come from external teams or automation. Instead, we handle it safely within Terraform.

Using Locals in Terraform

Locals are one of those quietly powerful Terraform features that fix real-world problems. In production environments, you often repeat values such as environment names, project codes, tag sets, naming prefixes, and locations. If those values are typed slightly differently in different places, Terraform interprets them as different—and that’s how you get configuration churn (resources being recreated or modified for no useful reason).



Locals let you define a value once and reuse it everywhere inside the module:

```
locals {  
  env_canon  = lower(trimspace(var.env))  
  project_canon = lower(trimspace(var.project_name))  
  prefix     = "${local.project_canon}-${local.env_canon}"  
}  
name = "${local.prefix}-nsg"
```

Here we:

1. **Clean** the input (trimspace, lower)
2. **Store** the cleaned version in a local
3. **Build** a reusable composite value (prefix)
4. **Consume** that reusable value in resources

From then on, resources can use:

```
name = "${local.prefix}-nsg"
```

instead of each one trying to build its own naming convention.

Why Not Just Use Variables?

Variables are for values that come from outside the module (user input, CI/CD, tfvars). Locals are for values derived inside the module—cleaned, transformed, or composed.

You often can't "fix" the input upstream, so you must clean it downstream.

Locals are ideal because they:

- Are evaluated once at plan time
- Are available everywhere in the module
- Don't have to be passed around like variables

What Locals Are Good For



Normalizing messy input

If someone types Dev, DEV, or dev, we can normalise:

```
locals {  
  env_canon = lower(trimspace(var.env))  
}
```

All resources then use local.env_canon, not the messy version.

Centralizing tags

Instead of repeating tag maps on every resource:

```
locals {  
  base_tags = {  
    app = local.project_canon  
    env = local.env_canon  
  }  
}
```

Then apply once per resource:

```
tags = local.base_tags
```

Building naming conventions

Organisations often require naming standards.

Locals let you enforce them consistently:

```
locals {  
  prefix = "${local.project_canon}-${local.env_canon}"  
}
```

Why Locals Help Reduce Churn

Terraform tracks resources using the key in *for_each*. If that key changes—even by casing or spacing—Terraform treats it as a new resource and recreates it. A cleaning layer in locals ensures that all keys are lowercase, trimmed, and punctuation-normalized. Even if the input varies, the cleaned key remains consistent. That’s how locals make configurations deterministic and predictable.

Things Locals Are Not For

- Storing secrets (use variables and secret managers)
- Passing data between modules (use outputs)
- Replacing variables entirely—they sit between variables and resources as a transformation layer.

Validation and Guardrails

Once our inputs are clean and canonicalized, the next step is to make sure that only valid and safe values are ever applied. Terraform’s validation features form the second layer of defense: they don’t just normalize input, they actively reject it when it’s dangerous or inconsistent.

Variable Validation

Terraform allows you to attach validation rules directly to input variables. These rules check values before planning or applying, providing human-readable error messages when input

fails. This ensures that a developer cannot accidentally deploy with unsafe or non-compliant data.

Example – Block open inbound access:

```
variable "allowed_cidrs" {  
  type = list(string)  
  validation {  
    condition = alltrue([for c in var.allowed_cidrs : c != "0.0.0.0/0"])  
    error_message = "Public access (0.0.0.0/0) is not permitted."  
  }  
}
```

When a user tries to pass a public CIDR range, Terraform halts immediately and prints the validation error. This is an example of fail-fast behaviour: we stop invalid input before it can even reach the plan.

Preconditions and Postconditions

Preconditions and postconditions allow you to assert expectations at the resource or module level. A precondition runs before the resource is created; a postcondition runs after. Both help enforce contract-like guarantees.

Example – Enforce that each NSG rule name includes the environment:

```
resource "azurerm_network_security_rule" "allow_https" {  
  name          = "${local.prefix}-allow-https"  
  direction     = "Inbound"  
  access        = "Allow"  
  priority      = 200  
  protocol      = "Tcp"  
  source_address_prefixes = local.allowed_cidrs_clean  
  destination_port_range = 443  
  
  lifecycle {  
    precondition {  
      condition = can(regex("${local.env_canon}", self.name))  
      error_message = "Rule name must include environment identifier."  
    }  
    postcondition {  
      condition = self.access == "Allow"  
      error_message = "Unexpected rule type — only Allow rules are permitted in this module."  
    }  
  }  
}
```

```
}  
}
```

These guardrails make policies self-enforcing: naming and access patterns are guaranteed without needing a separate compliance tool.

Conditional Logic

Conditional checks in locals and expressions help your module adapt safely to varying input. Use functions like *can()* and *try()* to handle missing attributes gracefully and ternary logic to decide behaviour.

Example – Fallback when tag not provided:

```
locals {  
  owner_tag = try(var.tags["owner"], "unassigned")  
}
```

Example – Conditional count:

```
resource "azurerm_monitor_diagnostic_setting" "nsg_logs" {  
  count = var.enable_logging ? 1 : 0  
  name = "nsg-logs"  
  target_resource_id = azurerm_network_security_group.nsg.id  
  log_analytics_workspace_id = var.log_workspace_id  
}
```

Bringing It All Together

Hygiene and validation work best together. First, locals clean and normalise inputs so they are predictable. Then validations and conditions ensure that only acceptable values get through. Together they create Terraform code that is deterministic, reproducible, and policy-compliant.



Takeaways

- Normalize first, validate second — cleanliness before compliance.
- Fail early with clear messages — save time and prevent unsafe plans.
- Use locals for hygiene, validations for protection, and pre/postconditions for assurance.
- Aim for Terraform code that is self-auditing — it refuses to break policy even under human error.

Lab 2

Hygiene and Validation

In this lab you will learn how to control variability in Terraform configurations by normalising inputs, enforcing guardrails, and producing deterministic infrastructure behaviour.



Module 3: Dynamic Blocks & Deterministic Naming

Module Overview

This module builds directly on the work you completed in Modules 1 and 2. By this point, you already have a resource group and a network security group (NSG) whose rules are generated dynamically from cleaned input maps. Names are normalised, rules are sanitised, and the configuration behaves predictably even when inputs change order or casing.

In this module, you extend that design to networking. You will introduce a Virtual Network (VNet) and subnets, starting from a deliberately repetitive baseline and then refactoring to a map-driven, for_each-based design. The aim is to make the subnet and NSG-association layer clean and deterministic like the NSG rules.

Starting Point: Module 2 Exemplar

The starting code for this module is the exemplar from the end of Module 2, with one important addition: a simple Virtual Network and two subnets. The NSG configuration is already in good shape: rules are defined in `var.nsg_rules`, cleaned into `local.nsg_rules_clean`, and realised in `azurerm_network_security_group.nsg` via a dynamic "security_rule" block. Naming is centralised in `locals.tf` using a shared prefix, and tags are built from a small, consistent map.

To prepare for Module 3, the exemplar is extended with:

- an `azurerm_virtual_network.vnet` resource using `var.vnet_address_space`,
- two `azurerm_subnet` resources (web and app) that read from `var.subnet_cidrs`.

These subnets are intentionally defined as separate, repetitive resources so that you can practice designing a `for_each` pattern from a "works but repetitive" baseline.

Dynamic Blocks and `for_each`: Recap

You have already seen dynamic blocks and `for_each` in action in the NSG configuration. There, a map of rule definitions is normalised in locals and then passed to a dynamic `security_rule` block with `for_each = local.nsg_rules_clean`. This ensures that rule identities are based on deterministic keys rather than input order.

The same idea applies to subnets. Rather than defining one resource per subnet, you treat the set of subnets as data and let Terraform generate the corresponding resources. The trick is to choose stable keys and to normalise them so that accidental formatting differences do not cause churn.

Normalizing Subnet Inputs

The first step in refactoring the subnet layer is to normalise the subnet input map. The `terraform.tfvars` file contains a `subnet_cidrs` map that looks like this:

```
subnet_cidrs = {
  web = "10.0.1.0/24"
  app = "10.0.2.0/24"
}
```

These values are structurally sound, but the keys might not remain perfectly formatted over time. To protect against this, you define `subnet_cidrs_clean` in `locals.tf`:

```
subnet_cidrs_clean = {
  for name, cidr in var.subnet_cidrs :
    lower(trimspace(name)) => trimspace(cidr)
}
```

This expression lower-cases and trims each subnet name and removes stray spaces from the CIDR values. The result is a deterministic map suitable for use as a `for_each` source.

Refactoring Subnets to `for_each`

Once `subnet_cidrs_clean` is in place, you can replace the two separate subnet resources with a single resource that iterates over the cleaned map:

```
resource "azurerm_subnet" "subnet" {
  for_each      = local.subnet_cidrs_clean

  name                = "${local.prefix}-subnet-${each.key}"
  resource_group_name = azurerm_resource_group.rg.name
  virtual_network_name = azurerm_virtual_network.vnet.name
  address_prefixes    = [each.value]
}
```

This refactor moves subnet definition into data. Each entry in `subnet_cidrs_clean` produces one subnet whose name and `address_prefixes` are derived from the key and value. Adding a new subnet becomes a matter of editing `terraform.tfvars`; no new Terraform resources are required.

If you have already applied the baseline with separate web and app subnets, Terraform will treat this as a resource address change (destroying the old resources and creating new ones). In a production setting, you would consider a more careful state migration, but in the lab context the focus is on understanding the pattern.

Associating NSGs with Subnets

The next step is to ensure that each subnet is associated with the NSG. Rather than defining separate association resources for web and app, you reuse the same `for_each` pattern, this time iterating over the subnet resources themselves:

```
resource "azurerm_subnet_network_security_group_association" "subnet" {
  for_each           = azurerm_subnet.subnet

  subnet_id           = each.value.id
  network_security_group_id = azurerm_network_security_group.nsg.id
}
```

This design ensures that associations follow the subnets automatically. If you add a new subnet by updating `subnet_cidrs`, the corresponding association for that subnet appears in the plan without any additional Terraform code. Removing a subnet removes the association as well.

Testing Stability: Reordering and Extending

A good `for_each` design should be robust to ordering changes and straightforward to extend. You can test this by reordering entries in `subnet_cidrs` or by adding new ones. Reordering keys should not cause Terraform to replace resources; only adding or removing keys should have an effect.

For example, reordering `subnet_cidrs` as follows:

```
subnet_cidrs = {
  app = "10.0.2.0/24"
  web = "10.0.1.0/24"
}
```

should produce a zero-diff plan for subnets and associations, because the map keys remain the same. By contrast, adding a new subnet:

```
subnet_cidrs = {
  web  = "10.0.1.0/24"
  app  = "10.0.2.0/24"
```

```
data = "10.0.3.0/24"
}
```

should cause Terraform to propose a new `azurerm_subnet.subnet["data"]` and a matching `azurerm_subnet_network_security_group_association.subnet["data"]`.

Summary

Module 3 takes the principles of input hygiene and dynamic resource generation from Modules 1 and 2 and applies them to the networking layer. You start with a simple but repetitive VNet configuration and end with a clean, map-driven design where subnets and their NSG associations are generated automatically from normalised input data. This makes the configuration easier to extend and less prone to accidental state churn as requirements change.

Lab 3

Dynamic Blocks & Deterministic Subnets

In this lab you will learn how to evolve a naïve Terraform configuration into a deterministic, scalable design by centralising data and proving stability under change.



Module 4: Remote State and Multi-Environment Operation in Terraform

Module Overview

Up to this point in the course, Terraform has been used in a largely local context. State has been private, execution has been serial, and the consequences of mistakes have been limited to a single workstation. That model does not survive contact with real teams.

This module introduces the operational realities that emerge when Terraform moves from an individual tool to a **shared system of record**. It explains why state must be treated as shared infrastructure, why environments are defined by state rather than code, and why Terraform is deliberately strict about backend configuration.

This module is foundational. Every governance, promotion, and reconciliation pattern later in the course assumes that the ideas here are understood.

Why Local State Breaks Down

Local Terraform state works well when:

- a single person is working

- changes are infrequent
- mistakes are easy to undo

As soon as teams grow, those assumptions fail.

Common failure modes include:

- two people modifying the same infrastructure unknowingly
- state drift caused by lost or overwritten state files
- unclear ownership of shared resources
- accidental execution against the wrong environment

These failures are not caused by Terraform. They are caused by **humans sharing implicit state**. Remote state exists to make that shared state explicit, durable, and observable.

Remote State as Shared Infrastructure

When Terraform state is stored remotely, it stops being an implementation detail and becomes infrastructure in its own right.

Remote state provides:

- a single source of truth
- shared visibility
- locking to prevent concurrent modification
- durability beyond any one machine

More importantly, it introduces shared responsibility. Once state is shared, changes affect more than the person running Terraform. This is the first point in the course where Terraform becomes a team activity rather than a personal one.

The Backend Chicken-and-Egg Problem

Remote state introduces an unavoidable design tension. Terraform needs a backend in order to manage state. But the backend itself is infrastructure that must be created. This creates a chicken-and-egg problem that Terraform does not attempt to hide or automate away.

Instead, Terraform makes a deliberate design choice:

- backend infrastructure is created separately
- backend configuration is explicit and static
- backend bootstrapping is intentionally manual

This separation forces teams to think about trust, ownership, and lifecycle early — before automation hides those decisions.

Backend Bootstrap as a Deliberate Pattern

In practice, many teams formalise backend creation as a bootstrap phase. A bootstrap is a deliberately minimal Terraform configuration whose sole responsibility is to create and manage the remote backend infrastructure itself, such as storage accounts, containers, and access controls. Once established, this bootstrap is applied infrequently and changes only when backend requirements evolve.

The key characteristics of a bootstrap pattern are:

- it is intentionally simple and stable
- it has a narrow scope and limited blast radius
- it is treated as foundational infrastructure
- it is applied before any workload or environment state exists

Crucially, the bootstrap does not eliminate the chicken-and-egg problem — it makes it explicit and manageable. By separating backend lifecycle from workload lifecycle, teams reduce risk and avoid accidental coupling between state storage and the resources that depend on it. Different organisations implement bootstrap patterns differently. The underlying principle is consistent: state storage is critical infrastructure and deserves its own lifecycle.

Why Backend Configuration Is Treated Literally

One of the most surprising behaviours for new users is that Terraform treats backend configuration as **literal and immutable**.

Terraform does not:

- interpolate variables into backend configuration
- infer intent based on folder structure
- guess which environment you meant

This is not a limitation. It is a safety mechanism. Backend configuration defines **where authority lives**. Terraform requires that authority to be explicit, repeatable, and reviewable. Ambiguity at this layer would be catastrophic.

Environments Are Defined by State, Not Code

A common misconception is that environments are created by:

- folders
- workspaces
- variable files

In reality, environments are defined by **which state file Terraform is operating against**.

The state location — not the code — is the true environment boundary.

This distinction matters because:

- the same code can safely manage multiple environments
- changing variables does not change authority
- mis-targeting state is far more dangerous than mis-typing a value

Understanding this prepares learners for later modules where promotion, governance, and reconciliation all depend on correct state targeting.

Human Error Is the Primary Risk

Most Terraform incidents are not caused by bugs in Terraform.

They are caused by:

- running commands in the wrong directory
- targeting the wrong backend
- assuming context that is no longer true
- copying patterns without understanding boundaries

Terraform's strictness around backend configuration, initialisation, and reconfiguration exists to **surface these risks early**.

Guardrails in Terraform are not about convenience.

They are about forcing humans to slow down at the right moments.

Backend Reconfiguration as a Deliberate Act

Reconfiguring a backend is one of the most powerful actions a Terraform user can take.

It effectively tells Terraform:

“The authority boundary has changed.”

Because of this, Terraform requires reconfiguration to be explicit and intentional. It does not allow silent backend switching, and it caches backend configuration locally to prevent accidental drift.

These behaviours are frequently frustrating — until you understand what they are protecting.

Lightweight Guardrails: Verifying Your Backend Target

A recurring real-world failure mode in Terraform estates is simple but costly: running a plan or apply against the wrong state file. This mistake is rarely caused by Terraform itself. It is usually caused by humans working across multiple environments, directories, and credentials, where context is easy to assume and hard to verify.

Mature teams therefore introduce small, low-friction guardrails that confirm execution context before any state-changing operation. One common pattern is a backend key

checker: a lightweight script that reads Terraform's locally cached backend metadata and prints the backend key currently in use. The goal is not to replace discipline or approval workflows. It is to reduce accidental cross-environment mistakes by making the current state target visible at the moment it matters.

Key characteristics of this pattern:

- it is informational, not authoritative
- it is fast enough to run every time
- it relies on Terraform's own cached backend metadata
- it can be used interactively or embedded into pipeline preflight checks

In this course, the lab demonstrates a simple example of this idea using a helper script that prints the backend key Terraform is currently configured to use. Over time, organisations often evolve this pattern further, for example by enforcing expected key formats per environment and failing early on mismatch.

Scaling the Pattern Beyond the CLI

The patterns introduced in this module are not specific to local execution.

They are the foundation for:

- multi-environment CI/CD pipelines
- promotion workflows
- separation of duties
- governance enforcement

Automation amplifies both correctness and mistakes.

Understanding state boundaries before introducing pipelines is essential.

This module deliberately precedes any discussion of CI/CD.

How This Module Connects Forward

Everything that follows in the course assumes the mental models established here:

- Module 5 builds on deterministic refactoring and state identity
- Module 6 assumes clean environment separation
- Module 7 governs promotion across environments
- Module 8 reconciles long-lived state with central platforms

If Module 4 feels strict, that is intentional.

The strictness here prevents ambiguity later.

Module Summary

This module reframes Terraform state from a file into a **shared authority boundary**.

You learned that:

- local state does not scale to teams
- remote state is shared infrastructure
- backend bootstrap is intentionally separate
- environments are defined by state, not code
- Terraform's rigidity exists to protect humans

The accompanying lab places you inside these constraints and makes their consequences tangible. Understanding them is what allows Terraform to be used safely at scale.

Lab 4

Remote State and Multiple Environments

In this lab you will expand your configuration by introducing remote state and multi-environment structure



Module 5: Modularization, Contracts and Validations

Module Overview

Module 5 moves from writing well-structured flat Terraform to designing reusable, production-ready modules. You will refactor an existing configuration into two focused modules—security and network—while preserving behaviour.

By the end of this module, you should be able to design modules with clear contracts, strong validation, and zero-diff refactors.

Why Terraform Modules?

Terraform modules are the primary mechanism for reuse and encapsulation. They allow practitioners to package infrastructure patterns—such as a network layout or a security configuration—and consume them repeatedly in different environments or projects.

A well-designed module:

- Hides implementation detail behind a small, clear interface.
- Offers strong validation to catch misuse early.
- Produces predictable resources with stable names and addresses.
- Reduces clutter in the root configuration.

Module Boundaries: Security and Network

In this course we deliberately draw the module boundary between security concerns and network layout:

modules/security:

- Owns the NSG configuration.
- Generates NSG rules dynamically from a cleaned rule map.
- Exposes NSG ID and, optionally, other attributes via outputs.

modules/network:

- Owns the VNET and subnet layout.
- Generates subnets dynamically from cleaned input.
- Associates subnets with the NSG produced by modules/security.
- Exposes VNET and subnet identifiers.

This separation mirrors real-world patterns where network and security responsibilities are often split between teams, but must coordinate via clearly defined interfaces.

Refactoring a Flat Configuration

The starting point for this module is a working, flat Terraform configuration that already follows good hygiene and dynamic patterns from Module 3. It contains:

- A resource group.
- A dynamically generated NSG and its rules.
- A dynamically generated VNET and subnets.
- Associations that link subnets to the NSG.

Refactoring to modules should preserve behaviour. The primary task is to lift related resources into modules without changing their names, keys, or ForceNew fields. A successful refactor will therefore produce a plan with no changes.

Designing Module Contracts

A module contract consists of its inputs (variables) and outputs. Good module contracts:

- Use specific types instead of overly generic ones.
- Provide sensible defaults where appropriate.
- Clearly indicate which values are required.
- Include validation to block obvious misuse.

For example, the security module might require:

- prefix – the canonical name prefix used for all resources.
- nsg_rules – a cleaned map of rules that has already been canonicalized in locals.

The network module might accept:

- prefix – again, a canonical name prefix.
- address_space – the VNET CIDR.
- subnets – a map of subnet-name to subnet-CIDR.
- nsg_id – an NSG to associate with subnets (optional in some designs).

Validation and Guardrails at Module Level

Modules are a natural enforcement point for organisational policy. Validation blocks allow authors to express rules such as:

- VNET CIDRs must be valid CIDR strings.
- Subnet CIDRs must not overlap or use 0.0.0.0/0.
- Rule priorities must fall within a specific range.

example:

```
variable "address_space" {
  type = string
  validation {
    condition = can(cidrhost(var.address_space, 0))
    error_message = "The VNET address_space must be a valid CIDR."
  }
}
```

By failing fast with clear error messages, modules help teams detect issues before plans ever reach the cloud.

Wiring Modules from the Root

After extracting module logic, the root configuration becomes a thin orchestrator. It:

- Defines and cleans input data using locals (as established in Module 2 and 3).
- Calls modules/security and modules/network with the appropriate arguments.
- Passes outputs from one module into another where required (for example, NSG IDs into the network module).
- Remains small and easy to read.

example root wiring:

```

module "security" {
  source      = "../modules/security"
  prefix      = local.prefix
  resource_group_name = azurerm_resource_group.rg.name
  nsg_rules    = var.nsg_rules
}

module "network" {
  source      = "../modules/network"
  prefix      = local.prefix
  resource_group_name = azurerm_resource_group.rg.name
  address_space    = local.vnet_clean[var.env].address_space
  subnets         = local.vnet_clean[var.env].subnets
  nsg_id           = module.security.nsg_id
}

```

Zero-Diff Refactor Principle

A central teaching goal of this module is that refactors should not change behaviour. When you compare the plan from the original flat configuration with the modularized version, the ideal outcome is an empty diff (no changes).

If the refactored configuration does show differences, it suggests one of the following:

- Resource names differ from the original.
- Keys used in `for_each` have changed.
- A `ForceNew` argument (such as a name or CIDR) has been altered.
- A resource was inadvertently moved or recreated instead of preserved.

You are encouraged to use `terraform plan` as a refactor safety net—an automated check that the behaviour has not been accidentally modified.

Changing Addresses with `moved` and `state mv`

As established in Module 1, resource addresses form Terraform's identity model. In this module, that principle is applied to safe refactoring. Once a resource is in state, its address forms part of Terraform's identity for that object. Refactors such as renaming a resource, splitting it into modules, or restructuring modules can change the address. If Terraform is not informed about the move, it will typically plan to destroy the old resource and create a new one.

Terraform provides two mechanisms to handle address changes safely:

`moved` blocks, which live in configuration and describe address changes declaratively.

`terraform state mv`, which performs a one-off move directly in state.

The moved block is preferred for long-lived modules and libraries because it documents the change in version control and applies consistently across environments. The state mv command can be useful for ad hoc fixes or one-time transitions.

A typical moved block looks like:

```
moved {  
  from = azurerm_network_security_group.imported  
  to   = azurerm_network_security_group.app  
}
```

After adding the moved block, a terraform plan should show that the resource has been moved in state without being destroyed and recreated. Reviewing the plan carefully is part of the safety check for any state operation

Evolving Modules Over Time

In real environments, modules rarely remain static. They may grow to support additional features or be split into smaller units as complexity increases. The patterns introduced here—clear contracts, strong validation, and stable naming—form the basis for healthy module evolution.

When enhancing a module, practitioners should:

- Introduce new inputs in a backward-compatible way where possible.
- Avoid renaming resources unless there is a clear migration plan.
- Extend validation in a way that does not break existing, valid consumers.
- Consider versioning strategies (for example, using a private module registry).

Looking Ahead to Multi-Environment Use

Module 4 sets up the foundation for Module 5, where modules are used across multiple environments (dev, test, prod) with different tfvars and backends. The key idea is that environment awareness lives at the root level—modules themselves remain environment-agnostic.

This separation of concerns simplifies reasoning:

- Modules describe what a network or NSG looks like in general.
- Root configurations and pipelines decide where and when those modules are applied (which subscription, which region, which backend key, etc.).

Lab 5

Modularisation, Contracts and Validations

In this lab you will break a monolith into modules without changing deployed resources



Module 6 – Publishing and Consuming Terraform Modules

Module Overview

Up to this point in the course, Terraform has likely felt empowering. You can describe infrastructure clearly, repeatably, and safely. Once environments are created successfully, it is natural to believe that the hardest part is behind you.

In reality, the most difficult problems appear only after Terraform starts being reused.

This module exists to explore what happens when Terraform code is no longer written for a single moment in time, but must remain reliable across environments, teams, and months or years of change.

From Refactoring to Reuse

Many Terraform projects begin as a single root module. As they grow, refactoring into child modules improves readability and reduces duplication. While refactoring improves structure, it does not solve lifecycle problems. The moment a module is reused, it acquires consumers. Those consumers develop expectations about behaviour and stability. This is the moment where Terraform code changes character. It stops being a script and starts behaving like a shared system.

Modules as Products

When a Terraform module is consumed by more than one root or environment, it effectively becomes a product. A product is defined by expectations: that behaviour remains consistent, that changes are deliberate, and that breaking changes are rare and well-communicated. In Terraform, those expectations are expressed entirely through module inputs and outputs. These form the public contract of the module.

Understanding Module Contracts

A module contract is the agreement between a module and its consumers. It is defined by: the names and meanings of input variables, their types and defaults, and the names and structures of outputs. Terraform can validate syntax, but it cannot validate intent. A change that appears harmless inside a module can break every consumer if the contract is altered. This is why contract discipline matters more than internal implementation.

Roots and Responsibility

Root modules exist to express intent. They define which modules are used, which versions are consumed, and how those modules are wired together for a specific environment. Child modules exist to encapsulate behaviour. They should not attempt to infer environment-specific intent.

Keeping this separation clear allows environments to evolve independently and reduces the risk of unintended side effects.

Lab 6a

Module Foundations

In this lab, you will...

- Publish modules locally
- Publish modules remotely
- Consume published modules
- Control promotion by explicitly updating version pins
- Experience promotion as a deliberate human decision



Why Versioning Matters

Versioning exists to communicate risk and intent. A version number tells consumers whether a change is safe to adopt or requires careful consideration. Using floating references such as “latest” removes this signal. Upgrades become implicit rather than deliberate.

Pinning module versions ensures that change only occurs when someone explicitly decides it should.

Environments and Promotion

Different environments have different purposes. Development environments prioritise speed and experimentation. Production environments prioritise stability. Because of this, environments cannot always move in lockstep. Promotion becomes the act of deliberately moving a specific environment to a newer module version.

In Terraform, promotion is simply changing a version reference. The significance lies in the decision, not the mechanism.

Promotion Is a Human Decision

Terraform can calculate changes accurately, but it cannot decide whether those changes are appropriate. Reviewing a Terraform plan is a human responsibility. It is an opportunity to assess impact, risk, and timing before any infrastructure is modified.

Automation can execute decisions consistently, but it cannot replace judgement.

Automation and Its Limits

Pipelines are valuable for reducing execution errors and ensuring repeatability. However, automation does not create intent. Approval gates and pipelines can give a sense of safety without improving decision quality.

Good automation follows clear human decisions, rather than attempting to replace them.

Environment Ownership

When each environment owns its own promotion decisions, responsibility becomes clear. Environment-scoped pipelines allow environments to advance at appropriate speeds, without forcing unintended changes elsewhere.

This model aligns Terraform workflows with real organisational practices.

Lab 6b

Pipeline Automation

In this lab, you will...

- Deploy environments using a local pipeline
- Deploy environments using DevOps



Closing Thoughts

This module is not about learning new Terraform syntax. It is about understanding how Terraform systems behave over time. The goal is not speed, but trust. Not automation for its own sake, but deliberate control.

Module 7: Change Governance and Promotion Control in Terraform

Why This Module Exists

By the time Terraform is introduced into a real organisation, the problem is almost never “how do we create infrastructure?”

That problem was solved months — sometimes years — earlier.

The harder problem is this:

How do we allow change to happen safely, predictably, and visibly once infrastructure is already business-critical?

At this point, teams are no longer experimenting. They are operating production systems, under pressure, with real consequences for mistakes. Changes are reviewed, questioned, delayed, and occasionally rushed through in emergencies.

This module sits squarely in that reality. It explores how Terraform fits into organisations that care about *who is allowed to change what, when, and why* — and what happens when those answers are unclear.

When Informal Governance Stops Working

In small teams, governance tends to be implicit.

The same people:

- design the infrastructure
- approve the change
- apply the change

Trust is personal. Conversations happen at desks or in chat. When something goes wrong, the blast radius is usually contained.

As organisations grow, that model quietly collapses.

Suddenly:

- environments multiply
- teams specialise
- auditors ask questions
- production outages make the news

What used to be “just apply it” becomes a risk.

Terraform often enters at this stage, not as a silver bullet, but as a way to *formalise* how infrastructure changes are made. The mistake many teams make is assuming Terraform itself provides governance. It doesn’t!

Automation Is Fast — But It’s Not Wise

Terraform is excellent at answering one question: “*If I run this, what will change?*”

It is deliberately silent on another: “*Should this change happen at all?*”

Pipelines execute plans. Approvals allow execution. Neither understands business context, operational risk, or organisational intent. This distinction matters more than most teams expect.

Many governance failures start with good intentions:

- “Let’s automate promotions.”
- “Let’s enforce approvals.”
- “Let’s lock this down so nobody can break prod.”

Without a clear model of intent, these efforts quietly turn into bottlenecks, workarounds, or worse — false confidence.

Promotion Is Not a Technical Step

In mature environments, promotion stops being a technical concern and becomes a **statement of confidence**.

Promoting a change to a higher environment implicitly says:

- we expected this change
- we understand its impact
- we are willing to accept the risk

Terraform doesn’t know any of this. Pipelines don’t either. They only know *how* to execute, not *why*. This is why promotion must be explicit and deliberate. When promotion is inferred — from branch names, pipeline wiring, or convention — accountability becomes blurred. When something goes wrong, nobody is quite sure *who decided this was OK*.

Intent, Approval, and Execution Are Different Things

One of the most common anti-patterns in infrastructure automation is treating approval as intent. They are not the same.

Approval answers: “Are you allowed to do this?”

Intent answers: “Is this what we actually want to do?”

In well-designed systems:

- intent is declared openly
- approval confirms permission
- execution is mechanical

When these are conflated, approvals become rubber stamps and pipelines quietly turn into decision-makers — something they were never designed to be.

Governance Emerges from Layers, Not Tools

There is rarely a single “governance mechanism” in real systems.

Instead, governance emerges from layers:

- version control defines what *can* exist

- validation defines what is *acceptable*
- pipelines define *when* execution is possible
- identity defines *who* can act
- environments define *where* impact occurs

Each layer reduces risk in a different way. Terraform lives inside this ecosystem. It does not sit above it, and it cannot compensate for weaknesses elsewhere.

When Pipelines Are Asked to Do Too Much

CI/CD pipelines are often over-credited.

They are excellent at:

- consistency
- repeatability
- enforcement

They are terrible at judgement.

When pipelines are treated as governance engines, organisations slowly drift into dangerous territory:

- logic becomes implicit
- exceptions become ad hoc
- failures become surprising

Pipelines work best when they enforce decisions made *outside* the pipeline, not when they attempt to replace them.

How Governance Fails in Practice

Governance systems rarely fail because they are too weak. They fail because they are too rigid in the wrong places and too loose in others.

Typical failure patterns include:

- governance that cannot authorise its own evolution
- rules that block recovery during incidents
- credentials that silently bypass safeguards
- environments that blur instead of isolate risk

None of these failures are theoretical. They are common in organisations that equate “more rules” with “more safety”. Terraform has a useful property here: it makes these failures visible.

Trust Boundaries and Where Power Really Lives

A hard-won realisation in mature Terraform estates is that authority always lives somewhere outside the code.

In practice:

- Terraform enforces only what the platform allows
- Git enforces only what identity allows
- pipelines enforce only what their credentials allow

This is not a weakness of Terraform. It is a reality of any system. Infrastructure as Code reduces accidental change. It does not prevent intentional authority. Governance design begins with acknowledging this.

Identity Is Not Just Authentication

Identity systems are often treated as plumbing. In reality, they define the outer boundary of governance. Over-privileged identities quietly undermine every other control. Long-lived credentials turn temporary trust into permanent risk.

Modern identity approaches tighten this boundary by making authority:

- contextual
- time-bound
- auditable

They don't remove trust — they make it visible.

Governance-of-Governance: The Uncomfortable Question

Every mature organisation eventually encounters a question it cannot ignore: *Who is allowed to change the rules?*

There is no technical solution to this. Only design choices.

Governance-of-governance is about acknowledging that ultimate authority exists, then making it:

- explicit rather than hidden
- constrained rather than absolute
- observable rather than informal

Terraform doesn't remove power. It helps reveal where it actually sits.

Break-Glass Is Not Failure — It's Honesty

Real systems need escape hatches. Incidents don't wait for approvals. Outages don't respect pipelines. Break-glass access exists because reality demands it.

The difference between mature and immature systems is not whether break-glass exists, but how it is treated:

- rare, not routine
- logged, not silent
- reviewed, not ignored

The most dangerous systems are the ones that claim they don't need it.

Plans as Signals, Not Just Previews

At this level, Terraform plans stop being “what will change” tools and start becoming **signals**.

They signal:

- risk
- governance friction
- misaligned expectations

A plan that is constantly blocked is telling you something. A plan that bypasses everything is telling you something else. Mature teams listen.

Closing the Loop

By the end of this module, Terraform should feel less like a deployment tool and more like a conversation between intent, authority, and automation. Nothing here eliminates human responsibility. Nothing here guarantees safety.

What it does provide is clarity:

- about who decides
- about who executes
- about where power lives
- and about how failures become visible

The lab that accompanies this module does not introduce new ideas. It simply removes the safety net and lets these realities surface. That is the point.

Lab 7 Change Governance	In this lab, you will... • Classify change • Authorise change • Govern change	
---	--	---

Module 8: Consuming and Reconciling Workloads with a Centralised Landing Zone

By this stage of the course, your Terraform configuration has evolved into a modular, multi-environment architecture using a remote Azure Storage backend. Earlier modules focused on creating clean patterns for new infrastructure and understanding how Terraform plans and applies change.

Building on the refactoring techniques introduced in Module 5, this module focuses on why address stability matters in long-lived estates rather than how to implement it, from *building* infrastructure to *operating Terraform safely in a mature estate*. This reflects a common organisational reality: Terraform is no longer new, the cloud estate is no longer empty, and infrastructure decisions made in the past must now be understood, governed, and reconciled rather than ignored.

Module 8 explores how Terraform is used to bring existing resources under management, how configuration can be refactored safely over time, and how workloads adapt once a centrally managed Landing Zone becomes authoritative.

Why Terraform State Operations Matter

Terraform state is the mechanism that links configuration to real-world infrastructure.

In greenfield environments, Terraform typically creates everything it manages, and state evolves naturally with each apply. In brownfield or long-lived environments, this assumption no longer holds. Teams often need to adopt Terraform for infrastructure that already exists, or reorganise configuration as systems grow and mature.

These scenarios require deliberate state operations, such as:

- importing resources that pre-date Terraform
- refactoring configuration by renaming resources or introducing modules
- changing resource addresses without triggering destruction
- operating safely with remote backends, shared access, and locking

Without an understanding of how Terraform state behaves, many legitimate refactors appear to Terraform as destroy-and-recreate operations. Learning how to manipulate state safely is therefore a core skill for advanced Terraform users and a central theme of this module.

Terraform State as an Ownership Boundary

Terraform does not manage infrastructure simply because it exists in Azure.

Terraform manages infrastructure **only when it is present in state**.

Each Terraform state file represents an ownership boundary. Within that boundary, Terraform is authoritative and is expected to fully manage the lifecycle of the resources it tracks. Outside that boundary, Terraform has no authority and must not be trusted to act.

This distinction is critical in multi-team environments:

- Azure visibility does not imply Terraform ownership
- losing or discarding state severs Terraform's authority
- importing resources is the mechanism by which ownership is recovered

HashiCorp intentionally designs Terraform to minimise cross-state dependencies. This reinforces clear responsibility boundaries and avoids implicit coupling between teams.

Importing Existing Infrastructure into Terraform State

The terraform import command allows an existing cloud resource to be associated with a Terraform resource address. After import, Terraform treats the resource as if it had been created by a previous apply.

Import is a foundational capability when onboarding Terraform into environments with pre-existing infrastructure.

Key characteristics of import:

- the resource must already exist in Azure
- configuration must exist at the target Terraform address
- both the Terraform address and provider resource ID are required
- a terraform plan must be run immediately afterwards

Import establishes **ownership**, not correctness.

If configuration does not fully match the existing resource, Terraform will propose changes. This may be intentional, but it may also indicate that the configuration needs adjustment before applying.

Generated Configuration Is a Starting Point, Not a Design

Terraform can generate starter configuration by inspecting existing resources. This can accelerate adoption but does not replace architectural judgement.

Generated configuration:

- reflects observed state, not desired design
- may include invalid or unconfigurable arguments
- may not align with Terraform best practices

Human review is required to:

- remove invalid attributes
- restructure resources appropriately
- replace hard-coded identifiers with references

This review phase is expected and normal in enterprise Terraform usage.

Refactoring Without Resource Re-creation

Once a resource exists in state, its Terraform address becomes part of its identity. Refactors such as renaming resources, reorganising modules, or restructuring configuration change that address.

If Terraform is not informed of the change, it will typically plan to destroy the old resource and create a new one.

As introduced in Module 5, Terraform provides `moved blocks` and `terraform state mv` to preserve identity during refactors. In long-lived estates, these tools become essential operational safety mechanisms rather than refactoring conveniences.

After any address change, reviewing the plan carefully is a mandatory safety step.

Remote Backends and Operational Safety

Remote backends store state centrally and enable safe collaboration, but they introduce operational considerations that must be understood.

Key behaviours include:

- backend configuration is cached locally in `.terraform/terraform.tfstate`
- this cache stores backend metadata, not resource state
- `terraform init -reconfigure` updates the cached backend
- plans and applies use the cached backend unless reconfigured

If the cached backend key does not match the intended environment, Terraform may modify the wrong state file. Understanding this behaviour is essential for safe multi-environment operation.

Centralisation and the Platform / Workload Split

As cloud adoption matures, organisations typically centralise foundational infrastructure into a managed Landing Zone, aligned with Azure's Cloud Adoption Framework.

In CAF-aligned environments:

- platform teams own networking, security, and governance
- workload teams consume platform capabilities
- platform resources exist in platform-managed Terraform state

Terraform enforces this separation through state ownership boundaries. Workload teams must not import, modify, or destroy platform resources.

Centralisation changes the rules. Teams move from designing infrastructure to consuming platform services.

Architectural Choices When Consuming a Landing Zone

When consuming a centrally managed Landing Zone, two technically valid approaches exist:

1. referencing Azure resources using data sources
2. reading outputs from platform state using `terraform_remote_state`

Both are supported by Terraform, but they carry different architectural implications.

Using Azure data sources:

- preserves clear ownership boundaries
- avoids implicit runtime coupling between teams
- aligns with CAF platform immutability principles

While `terraform_remote_state` can be useful when states share ownership and lifecycle, it introduces implicit dependencies that Terraform does not govern. In regulated or multi-team environments, this coupling is typically discouraged.

For this reason, the course adopts **Azure data sources as the primary consumption pattern**.

Consuming the Centralised Landing Zone

Once a Landing Zone exists, organisations usually define a clear forward-looking rule:

All new workloads must consume the centrally provided platform.

In practice:

- platform resources are treated as authoritative
- workloads deploy into existing networks and security boundaries
- platform constructs are referenced, not redefined

This phase prevents further divergence but does not resolve historical inconsistency.

Regaining Control of Existing Workloads

Existing workloads present a more complex challenge. Production systems are rarely rebuilt simply to achieve architectural consistency.

At this stage, organisations focus on:

- identifying workload-owned resources
- separating workload assets from platform assets
- restoring visibility and accountability

This phase is about ownership, not remediation. No assumptions are made about what must change. Instead, organisations create the conditions required to make informed decisions later.

Reconciling Workloads with the Platform Model

With ownership restored, reconciliation can begin.

Workloads are assessed against platform expectations:

- can networking be realigned safely?
- can security controls be inherited?
- which changes are in-place vs destructive?
- which changes require approval or downtime?

In practice, this assessment often leads to a replace-then-retire approach, where Landing Zone-aligned resources are introduced first and non-compliant legacy resources owned by the workload are explicitly retired once risk and impact are understood.

Terraform does not hide these decisions. It surfaces them clearly through the plan.

Rather than forcing immediate remediation, reconciliation often results in:

- phased migration plans
- documented risks
- accepted technical debt
- deferred remediation aligned to business priorities

Terraform acts as a **decision-support tool**, not an auto-remediation engine.

Applying the Model in Practice

A typical organisational journey follows these phases:

Phase 1 – Landing Zone Established

A platform-managed Landing Zone exists. Workloads still operate using bespoke patterns.

Phase 2 – Consume the Landing Zone

New workloads deploy into the Landing Zone using Azure data sources.

Phase 3 – Import Existing Workload Resources

Legacy workload resources are imported into Terraform state to restore visibility.

Phase 4 – Reconcile Networking and Security

Terraform plans surface which changes are safe, destructive, or high-risk.

Phase 5 – Governance Discussion

Destructive changes are reviewed, approved, deferred, or rejected based on business context.

Lab 8

Landing Zone Consumption

In this lab you will adopt and reconcile with a centrally governed Landing Zone



Module Summary

This module demonstrated Terraform state operations in the context of a realistic, multi-environment Azure estate.

You learned how to:

- recover ownership of existing infrastructure
- refactor configuration safely
- consume a centrally managed platform
- reconcile legacy workloads deliberately
- use Terraform plans as governance artefacts

These skills are essential for operating Terraform in brownfield environments and for evolving infrastructure over time without unnecessary disruption.

Appendix A - Bloom's Taxonomy Alignment

This course is intentionally designed to operate at the **upper levels of Bloom's Taxonomy**, consistent with **Level-7 (postgraduate / senior professional)** learning expectations.

Rather than focusing on recall or mechanical application, the course emphasises analysis, evaluation, and synthesis in realistic enterprise contexts.

Bloom's Taxonomy Alignment

Remember & Understand

(Lower levels – intentionally minimised)

Students demonstrate understanding of:

- Terraform state and resource identity
- Core language constructs (for_each, maps, locals, validation)
- Backend mechanics and environment separation

These concepts are introduced early (Modules 1–3) and treated as **foundational prerequisites**, not end goals.

Apply

(Selective, structured use)

Students apply knowledge to:

- Refactor flat configurations into modules
- Implement deterministic naming and hygiene
- Configure multi-environment backends
- Use validation and guardrails effectively

Application is always contextualised and constrained to reinforce *safe* patterns rather than experimentation for its own sake.

Analyse

(Core emphasis)

From Module 4 onwards, students are consistently required to:

- Analyse Terraform plans for unintended consequences
- Identify sources of churn, drift, and replacement risk
- Distinguish between configuration errors and design flaws
- Assess how state boundaries influence authority and ownership

Analysis is reinforced through zero-diff refactors, preflight checks, and plan-based reasoning.

Evaluate

(Strong emphasis)

Students are expected to evaluate:

- Whether a proposed change should be promoted
- Whether automation is appropriate or premature
- The risk and impact of destructive vs in-place changes
- The trade-offs between coupling and independence (e.g. data sources vs remote state)

Evaluation is explicit in Modules 6–8, where Terraform is treated as a decision-support system rather than an execution engine.

Create / Synthesize

(Capstone level)

In the later modules, students synthesize multiple concepts to:

- Design safe promotion workflows
- Reconcile existing workloads with platform models
- Establish governance-aware Terraform patterns
- Construct defensible infrastructure change processes

This synthesis reflects real-world problem solving, where there is no single “correct” solution—only informed, justifiable decisions.

Level-7 Learning Outcome Alignment

At Level 7, learners are expected to demonstrate:

Critical understanding of complex systems

This course requires students to reason about Terraform as part of a broader socio-technical system involving teams, environments, governance, and risk.

Independent judgement and decision-making

Students are not told *what* to deploy. They are taught how to assess impact, interpret plans, and decide whether change is appropriate.

Application of theory to ambiguous, real-world scenarios

Brownfield reconciliation, promotion governance, and platform consumption are deliberately ambiguous by design. Students must apply principles, not follow recipes.

Responsibility and professional accountability

The course reinforces that infrastructure automation carries real consequences. Terraform is positioned as a tool that amplifies human intent—good or bad—making responsibility unavoidable rather than abstract.

Summary

This course deliberately operates beyond procedural Terraform usage. It develops practitioners who can **analyse, evaluate, and govern infrastructure change** in complex environments.

The learning outcomes align with senior professional and postgraduate expectations, making the material suitable for:

- enterprise upskilling

- platform engineering education
- regulated industry contexts
- Level-7 or equivalent professional development programmes

Appendix B – Level-7 / UK Apprenticeship & HE Mapping

Level-7 Alignment (UK Apprenticeship & Higher Education Frameworks)

Frameworks Referenced

This course aligns with expectations set out in:

- **UK Framework for Higher Education Qualifications (FHEQ) – Level 7**
- **Institute for Apprenticeships (IfATE) – Level 7 Knowledge, Skills & Behaviours (KSBs)**
- **QAA Subject Benchmark guidance for Computing and Digital Technologies**

The mapping below demonstrates how the course meets **Level-7 cognitive demand**, professional autonomy, and applied responsibility.

Level-7 Descriptor Alignment (FHEQ)

FHEQ Level 7 Descriptor (Summary)

At Level 7, learners are expected to demonstrate:

- systematic understanding of complex bodies of knowledge
- critical awareness of current practice and constraints
- originality in application
- autonomous decision-making
- responsibility for professional practice

How This Course Meets Level-7 Expectations

1. Systematic Understanding of Complex Knowledge

Students develop a deep, structured understanding of Terraform as:

- a state-driven system
- an ownership and authority model
- a governance and risk surface

Rather than isolated commands or recipes, Terraform is taught as an **interacting system of design decisions**, spanning state, lifecycle, environment boundaries, and organisational constraints.

This satisfies Level-7 expectations of *systematic conceptual mastery*, not surface familiarity.

2. Critical Awareness of Practice and Constraints

The course explicitly addresses:

- brownfield infrastructure realities
- multi-team and regulated environments
- platform vs workload ownership boundaries
- organisational risk, blast radius, and governance

Students are not presented with idealised scenarios. Instead, they are required to reason within **realistic constraints**, reflecting current professional practice.

This aligns directly with FHEQ expectations around *critical awareness of real-world complexity*.

3. Originality in Application

Learners are repeatedly required to:

- design their own module boundaries
- decide when refactoring is safe or unsafe
- assess reconciliation options for existing workloads
- justify architectural choices rather than follow prescriptive steps

There is no single “correct” solution in later modules. Originality emerges through **justified design decisions**, not novel syntax.

This meets Level-7 criteria for originality through applied reasoning.

4. Autonomous Decision-Making

The course deliberately avoids fully automated outcomes.

Students must decide:

- whether a change should be promoted
- whether risk is acceptable
- whether remediation should be deferred
- how governance constraints should be applied

Terraform plans are used as **decision-support artefacts**, reinforcing learner autonomy rather than tool-led determinism.

This strongly satisfies Level-7 expectations around independence and judgement.

5. Professional Responsibility and Accountability

Throughout the course, learners are positioned as:

- accountable for infrastructure change
- responsible for interpreting impact
- aware of organisational consequences

Automation is framed as an execution mechanism, not a substitute for responsibility.

This directly aligns with Level-7 expectations that learners demonstrate **professional accountability**, not just technical competence.

Alignment to Level-7 Apprenticeship (IfATE – KSBs)

Knowledge (K)

The course develops advanced knowledge of:

- infrastructure as code operating models
- state management and lifecycle control
- governance, promotion, and ownership models
- platform engineering principles

Knowledge is contextual, evaluative, and applied — consistent with Level-7 apprenticeship expectations.

Skills (S)

Learners demonstrate skills in:

- analysing Terraform plans for risk and impact
- refactoring live configurations safely
- reconciling legacy infrastructure with platform standards
- designing governance-aware automation patterns

Skills are demonstrated through **judgement-led tasks**, not procedural repetition.

Behaviours (B)

The course explicitly reinforces Level-7 professional behaviours, including:

- risk awareness
- accountability for decisions
- collaboration across ownership boundaries
- ethical handling of production systems

These behaviours are embedded in the narrative and assessment approach, not treated as optional soft skills.

QAA Computing Benchmark Alignment

The course aligns strongly with QAA expectations for advanced computing education by:

- integrating theory and professional practice
- addressing socio-technical systems, not isolated tooling
- requiring evaluation of trade-offs and constraints
- emphasising governance, sustainability, and operational safety

Terraform is treated as part of a broader **digital systems lifecycle**, consistent with postgraduate computing benchmarks.

Summary Statement (For Accreditation / QA Use)

This course meets UK Level-7 expectations by developing learners' ability to analyse, evaluate, and govern complex infrastructure systems autonomously. It moves beyond procedural tool usage to cultivate professional judgement, accountability, and decision-making in realistic enterprise contexts, consistent with FHEQ Level-7 descriptors, IfATE apprenticeship standards, and QAA benchmarking guidance.

Copyright

© 2026 Michael Coulling-Green QA. All rights reserved. These lab materials are provided as part of an authorised training course. Course attendees may reuse these materials for their own personal learning and reference outside of the training session. Unauthorised copying, redistribution, publication, or use of these materials to deliver training is prohibited without prior written permission from the author.